Operating Systems Syllabus Mapping & Viva Revision Guide

High-Performance Multi-Threaded Parallel K-Means Engine

Hardware Tested: 13th Gen Intel Core i5-13420H (12T)

OS & Platform: Linux Ubuntu 24.04 (WSL2)

Standards: C11, POSIX Threads (pthreads), -O3

Master Syllabus-to-Project Mapping Table

Use this table to connect textbook concepts directly to our implementation files when answering viva questions.

OS UNIT	TEXTBOOK TOPIC	WHERE & HOW IT IS USED IN YOUR PROJECT	RELEVANT FILES
Unit 1	**System Calls vs User Space**	Avoiding repeated `clone()` / `pthread_create` kernel context switches by maintaining a persistent Thread Pool across iterations.	`thread_pool.c`
Unit 1	**Cache & Memory Hierarchy**	64-byte L1 Cache Line alignment (`posix_memalign`) and MESI cache coherence false sharing resolution.	`accumulator.c` `aligned_memory.c`
Unit 2	**Thread Libraries (Pthreads)**	POSIX threads (`pthread_t` , `pthread_create` , `pthread_join`) 1:1 kernel thread execution model.	`thread_pool.c`
Unit 2	**IPC: Shared Memory Systems**	All threads share contiguous heap arrays (dataset points and centroids) with zero memory copy overhead.	`kmeans.c`
Unit 3	**CPU Scheduling & Stragglers**	Static chunk distribution vs. Dynamic FCFS Task Queue work-pulling to balance varying CPU core latencies (P-cores vs E-cores).	`task_queue.c` `benchmark_scheduler.c`
Unit 3	**Granularity & Parallel Cut-Off**	The Grain Size Problem: Task queue & thread wakeup overhead vs compute time (Empirical Sequential Cut-off).	`kmeans.c` `threshold_benchmark.c`
Unit 3	**Critical-Section Problem**	Guarding shared queue cursors (`head` , `tail` , `count`) using `pthread_mutex_t` mutual exclusion.	`task_queue.c`
Unit 3	**Producer-Consumer (Bounded Buffer)**	Bounded circular ring buffer pattern with Mutex and 2 Condition Variables (`not_empty` , `not_full`).	`task_queue.c`

OS UNIT	TEXTBOOK TOPIC	WHERE & HOW IT IS USED IN YOUR PROJECT	RELEVANT FILES
Unit 3	**Phase Barrier Synchronization**	Iteration barrier implemented via `pool_wait_iteration` using `pthread_cond_broadcast` on `all_done`.	`thread_pool.c`
Unit 4	**Deadlock Prevention**	Breaking Coffman's "Circular Wait" and "Hold-and-Wait" through a strict Single-Lock Hierarchy.	`task_queue.c` `thread_pool.c`
Unit 5	**Contiguous Allocation & Alignment**	Row-major contiguous dataset storage for spatial locality prefetching; 64-byte aligned allocation via `posix_memalign`.	`aligned_memory.c`
Unit 6	**UNIX Case Study & File Systems**	Parsing `/proc/cpuinfo` in the UNIX virtual pseudo-filesystem to detect CPU hardware model and logical threads dynamically.	`threshold_benchmark.c`

Detailed Unit Breakdown & Viva Voce Answers

UNIT 1: OS Architecture, Structures & Hardware Interaction

Core Concepts: System Calls vs User Space, Kernel Transitions, Hardware Cache Hierarchy, Spatial Locality, and the MESI Protocol.

- **User Mode vs Kernel Mode:** In naive multi-threaded K-Means, spawning and destroying threads in every clustering iteration forces thousands of `clone()` system calls and CPU ring-0 context switches. Our Thread Pool initializes P threads once and keeps workers executing in user-space.
- **Hardware Cache Lines & False Sharing:** The CPU fetches data from RAM in 64-byte blocks. When two cores concurrently write to adjacent accumulators on the same 64-byte line, the MESI protocol invalidates the line across peer cores, causing extreme interconnect bus traffic. We resolved this via `posix_memalign` and bitwise padding `(size + 63) & ~63`.

❓ Viva Question 1: "What hardware-level property did you optimize for in your thread accumulators?"

Ideal Answer: "We optimized for the **CPU L1 Cache Line size**, which is 64 bytes on modern x86-64 and ARM processors. In our multi-threaded clustering engine, each worker thread updates a local accumulator. If these accumulators are tightly packed, adjacent threads write to the same 64-byte cache line. This triggers **False Sharing** under the hardware **MESI cache coherence protocol**, forcing CPU cores to repeatedly invalidate each other's caches. We solved this by using `posix_memalign` to guarantee 64-byte boundary addresses and applied bitwise padding `(size + 63) & ~63` so each thread's buffer occupies its own independent cache lines, delivering a **5x to 6x empirical speedup**."

❓ Viva Question 2: "Why is creating threads per iteration bad from an OS architectural perspective?"

Ideal Answer: "Spawning threads repeatedly invokes the `clone()` system call in Linux. Every system call triggers a **hardware trap from User Mode to Kernel Mode**, page table allocations for thread stacks, and kernel scheduler accounting. By maintaining a persistent worker Thread Pool, threads are created once at startup and reused across all iterations, saving thousands of costly kernel transitions."

UNIT 2: Process Management, Threads & Inter-Process Communication

> **Core Concepts:** Process vs Thread, POSIX Thread Library (Pthreads), 1:1 Threading Model, and Shared Memory IPC.
>
> - **Thread Model:** We implement POSIX Threads (`pthread_t`) under the 1:1 kernel thread model. Each worker thread is an independent kernel schedulable entity with its own CPU registers and private stack, while sharing the heap.
> - **Shared Memory:** The multi-megabyte input point array and centroid coordinates live in shared heap memory. All worker threads read directly from these buffers with zero memory duplication or IPC messaging overhead.

> ❓ **Viva Question 3: "Why choose multi-threading over multi-processing (`fork()`) for this clustering project?"**
>
> **Ideal Answer:** "Threads within the same process share the virtual address space (heap). Because K-Means requires all workers to read the same 64 MB dataset points and shared cluster centroids on every iteration, threads allow **zero-copy direct memory access**. Using `fork()` would spawn separate processes with isolated address spaces, requiring either copy-on-write page fault overhead or explicit POSIX shared memory primitives (`shmget` / `mmap`)."

UNIT 3: CPU Scheduling & Process Synchronization (Most Critical)

Core Concepts: Scheduling Criteria, Straggler Mitigation, The Grain Size Problem, Critical Sections, Producer-Consumer Bounded Buffer, and Barriers.

- **Dynamic Work-Pulling vs Static Partitioning:** Under static scheduling (N/P), if one thread encounters operating system jitter or runs on an Intel Efficiency-core (E-core), all other threads sit idle at the barrier. With our Dynamic Ring Buffer Queue, points are divided into smaller chunks; faster threads pull additional chunks automatically, minimizing the Load Imbalance Ratio.
- **The Parallel Overhead Break-Even Cut-Off ($N_{\text{threshold}}$):** On small datasets ($N < 5,000$), the time spent acquiring queue mutex locks, waking threads via condition variables, and reducing accumulators exceeds the actual computation time, causing multi-threading to run up to 14x slower than single-threaded code. Our `kmeans_adaptive_run` dynamically switches to sequential execution below this cut-off.
- **The Critical-Section Problem:** Task queue indices (`head` , `tail` , `count`) are guarded by `pthread_mutex_t lock` .
- **Classic Producer-Consumer Pattern:** The task queue implements the classic Bounded Buffer pattern using two condition variables (`not_empty` and `not_full`). Consumers sleep without busy-waiting when empty.
- **Lock-Free Distance Computation:** Inner compute loops have **zero mutex locks**. Threads accumulate into private padded buffers, and synchronize only once at the iteration barrier via `pool_wait_iteration` .

❓ Viva Question 4: "How did you solve the Producer-Consumer problem in your Task Queue?"

Ideal Answer: "We implemented a bounded circular ring buffer in `task_queue.c` guarded by a mutex and two condition variables: `not_empty` and `not_full` . The main thread acts as the **Producer** (`queue_push`), and worker threads act as **Consumers** (`queue_pop`). When the queue is empty, worker threads sleep on `pthread_cond_wait(&q->not_empty, &q->lock)` without burning CPU cycles, and are signaled as soon as tasks are submitted."

❓ Viva Question 5: "Your teacher mentioned a threshold below which parallel computing becomes overhead. What causes this, and how does your code handle it?"

Ideal Answer: "This is known in OS CPU scheduling as the **Grain Size Problem**. The total parallel execution time is:
`T_parallel = (T_compute / P) + T_overhead`
Where `T_overhead` includes thread dispatch, task queue mutex locks, condition variable wakeups, and accumulator reduction. When the dataset size N is small (e.g. $N < 5,000$), T_{compute} is under 0.5 ms, making T_{overhead} dominate and causing multi-threaded speedup to drop to **0.1x – 0.5x (a severe slowdown)**.
Our empirical benchmark on an Intel i5-13420H revealed exact break-even crossover points: **$N \approx 5,000$ for 2 threads**, **$N \approx 10,000$ for 4 threads**, and **$N \approx 25,000$ for 12 threads**.
To solve this, our `kmeans_adaptive_run()` function inspects N: if $N < \text{threshold}$, it automatically bypasses multi-threading and runs single-threaded sequential execution, guaranteeing zero overhead across all input sizes."

❓ **Viva Question 6: "Why don't your worker threads lock a mutex when updating cluster centroids?"**

Ideal Answer: "Because acquiring a mutex inside the inner distance assignment loop (evaluated 320,000,000 times) causes catastrophic lock contention. In our benchmark, a naive global mutex ran in **1,868 ms**, whereas our lock-free thread-local accumulator architecture ran in **163 ms (11.4x faster)**. Each thread accumulates into its own private 64-byte padded memory, and the main thread reduces partial sums once at the iteration barrier."

UNIT 4: Deadlocks

> **Core Concepts:** System Model, Coffman Conditions (Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait), and Deadlock Prevention.
>
> - **Deadlock Prevention via Single-Lock Hierarchy:** Our system enforces a strict rule: **no thread ever holds more than one lock simultaneously**. Workers hold `q->lock` while popping tasks, release it, execute compute unlocked, and hold `pool->done_lock` only when decrementing the pending counter. This mathematically breaks Coffman's **Hold-and-Wait** and **Circular Wait** conditions.
> - **Termination Liveness:** In `queue_close()`, setting `q->closed = true` is immediately paired with `pthread_cond_broadcast(&q->not_empty)`. This wakes any sleeping workers so they gracefully exit without hanging indefinitely.

❓ Viva Question 7: "How does your thread pool design guarantee that workers will never deadlock?"

Ideal Answer: "We guarantee deadlock-free execution by mathematically eliminating Coffman's **Hold-and-Wait** and **Circular Wait** conditions through a **Single-Lock Discipline**. No worker thread ever requests a second mutex while holding a first mutex. Queue operations use `q->lock`, and task completion accounting uses `pool->done_lock`. Furthermore, shutdown deadlock is prevented by broadcasting to all waiting condition variables during `pool_shutdown()`."

UNIT 5: Memory Management & Virtual Memory

> **Core Concepts:** Contiguous Memory Allocation, Spatial Locality, Paging, Alignment, and Virtual Memory Thrashing.
>
> - **Contiguous Row-Major Storage:** Data points are laid out in flat 1D arrays: `points[p * d + dim]`. This maximizes **spatial locality**—when core 0 accesses dimension 0, the hardware prefetcher pulls subsequent dimensions into the L1 cache in a single memory burst.
> - **64-Byte Hardware Alignment:** We call POSIX `posix_memalign` to guarantee that buffers start at physical memory addresses divisible by 64 (last 6 bits are `000000`).
> - **Stress Testing Without Thrashing:** Our benchmark tests up to 1,000,000 8-D points (64 MB). This exceeds the CPU's 16 MB L3 cache to thoroughly stress memory bandwidth while remaining well within physical RAM limits, preventing page faults and disk thrashing.

❓ Viva Question 8: "How does your memory layout exploit spatial locality and hardware prefetching?"

Ideal Answer: "Points are stored contiguously in row-major order. When a core evaluates Euclidean distance, the CPU hardware prefetcher recognizes the sequential stride and loads the remaining dimensions into L1 cache in a single 64-byte burst. Additionally, `posix_memalign` guarantees buffers start aligned on 64-byte hardware boundaries, preventing memory blocks from straddling cache line splits."

UNIT 6: UNIX Case Study & File Systems

> **Core Concepts:** UNIX Virtual File System (VFS), `/proc` Pseudo-Filesystem, and POSIX I/O System Calls.
>
> - **UNIX `/proc` Pseudo-Filesystem:** In `threshold_benchmark.c`, we dynamically open and parse `/proc/cpuinfo` to extract the CPU model name and core topology directly from the Linux kernel.
> - **POSIX File I/O:** Standard POSIX file streams (`fopen`, `fprintf`, `fclose`) record empirical latency measurements directly to CSV files (`threshold_results.csv`) for plotting and verification.

❓ Viva Question 9: "How did your project interact with the UNIX operating system interface?"

Ideal Answer: "We interacted with the UNIX kernel through the **`/proc` virtual pseudo-filesystem** by parsing `/proc/cpuinfo` to dynamically detect processor topology and core counts, and used POSIX system calls (`sysconf(_SC_NPROCESSORS_ONLN)`, `clock_gettime(CLOCK_MONOTONIC)`) and standard file I/O to persist empirical benchmarking results."

30-Second Viva Voce "Cheat Sheet"

- **False Sharing Solution:** 64-byte cache line alignment via `posix_memalign` + bitwise padding `(size + 63) & ~63`. (5x–6x speedup).
- **Small Workload Overhead:** The Grain Size Problem. Mutex queue locks and thread wakeups exceed compute time when $N < 5,000$. Solved with `kmeans_adaptive_run()`.
- **Lock Contention Elimination:** Thread-Local Padded Accumulators; zero mutexes inside distance calculation loops. (11.4x faster than global mutex).
- **Straggler Mitigation:** Dynamic work-pulling ring buffer queue absorbs latency fluctuations between fast/slow cores.
- **Deadlock Prevention:** Single-Lock Hierarchy; workers never hold more than one mutex at any time (breaks Coffman's Circular Wait).
- **Thread Model:** POSIX Threads 1:1 kernel model; persistent thread pool avoids repeated `clone()` system calls.
